

Taller IPtable

Integrantes:

Andres Castro Gonzalez

Julian Esteban Rincon Rodrigues

Kevin Nicolai Garcia Rodriguez

I. Objetivo del taller

Se plantea construir un pequeño laboratorio con tres máquinas virtuales — Windows 10/11, Ubuntu y Kali Linux en el que Ubuntu funciona como firewall, controlando mediante iptables el tráfico que circula entre Kali y Windows. El taller exige implementar un total de seis reglas: tres orientadas al tráfico dirigido hacia Windows y tres orientadas al tráfico dirigido hacia Kali.

Para Windows se debían implementar:

- Una regla para bloquear un puerto TCP específico.
- Una regla para bloquear tráfico ICMP (ping).
- Una regla para permitir únicamente el tráfico TCP hacia un puerto específico (bloqueando el resto).

Para Kali se debían implementar:

- Una regla para bloquear un puerto TCP específico.
- Una regla para bloquear tráfico UDP hacia un puerto específico.
- Una regla para permitir tráfico ICMP.

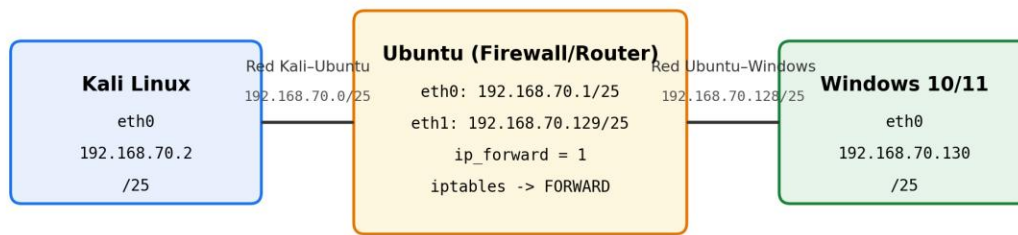
Cada regla debía probarse generando tráfico real desde Kali y desde Windows, verificando además que las reglas dirigidas a un nodo no afectaran el tráfico del otro, y capturando cada prueba con Wireshark para identificar direcciones IP, puertos, protocolo y banderas TCP (SYN, SYN/ACK, ACK o RST), comparando el comportamiento del tráfico permitido, bloqueado y rechazado.

II. Descripción del entorno y arquitectura de red

El laboratorio se construyó con tres máquinas virtuales sobre VirtualBox: Kali Linux, Ubuntu y Windows 10/11. A diferencia de un escenario con una única red compartida, aquí Ubuntu se configuró con dos interfaces de red, cada una conectada a una red privada distinta, de modo que Kali y Windows quedan en segmentos separados y sin ruta directa entre sí. Ubuntu es el único nodo con una interfaz en cada red, por lo que actúa necesariamente como router entre ambas.

Concretamente, el bloque 192.168.70.0/24 se dividió en dos subredes /25:

- Red Kali–Ubuntu: 192.168.70.0/25 (192.168.70.1 a 192.168.70.126). Ubuntu usa 192.168.70.1 en esta interfaz y Kali usa 192.168.70.2.
- Red Ubuntu–Windows: 192.168.70.128/25 (192.168.70.129 a 192.168.70.254). Ubuntu usa 192.168.70.129 en esta interfaz y Windows usa 192.168.70.130.



Kali y Windows no tienen ruta directa entre sí: todo el tráfico entre ambos atraviesa obligatoriamente Ubuntu, que actúa como router y aplica las reglas de iptables sobre la cadena FORWARD.

Fig. 1. Topología de red: Ubuntu conecta las dos subredes /25 y actúa como router/firewall entre Kali y Windows.

En Ubuntu se habilitó el reenvío de paquetes IP (`net.ipv4.ip_forward=1`), condición necesaria para que el sistema pueda enrutar tráfico entre sus dos interfaces en lugar de limitarse a recibir tráfico dirigido a sí mismo. Como consecuencia directa de esta topología, todo el tráfico entre Kali y Windows atraviesa obligatoriamente la pila de red de Ubuntu: si se desconecta o desactiva cualquiera de las dos interfaces de Ubuntu, Kali y Windows dejan de poder hacer ping entre sí, porque no existe ninguna otra ruta que los conecte. Esto es precisamente lo que permite que las reglas de filtrado se apliquen sobre la cadena FORWARD de iptables (el tráfico que Ubuntu reenvía) y no sobre INPUT u OUTPUT (que gobiernan el tráfico dirigido a Ubuntu mismo o generado por él).

Nodo	Interfaz	Dirección IP	Rol
Kali Linux	eth0	192.168.70.2 /25	Origen de pruebas (red Kali–Ubuntu)
Ubuntu	eth0	192.168.70.1 /25	Firewall / router – lado Kali
Ubuntu	eth1	192.168.70.129 /25	Firewall / router – lado Windows
Windows 10/11	eth0	192.168.70.130 /25	Destino de pruebas (red Ubuntu–Windows)

Antes de aplicar cualquier regla, se verificó la conectividad base: con la política por defecto de la cadena FORWARD en ACCEPT, un ping desde Kali llega a Windows y viceversa, atravesando Ubuntu sin ningún filtrado. Sobre esa base se fueron agregando, una por una, las seis reglas exigidas por el taller, probando cada una antes y después de aplicarla, y verificando adicionalmente que no interfiriera con el tráfico del nodo contrario.

III. Metodología de verificación

Para cada una de las seis reglas se siguió el mismo procedimiento en cuatro pasos:

- Antes de la regla: se genera el tráfico correspondiente y se documenta el comportamiento normal (sin firewall), típicamente un puerto “cerrado” que responde con RST (TCP) o Destination Unreachable (UDP/ICMP).
- Configuración: se agrega la regla en Ubuntu con `iptables -A FORWARD ...` y se verifica con `iptables -L -n -v`.

- Después de la regla: se repite exactamente la misma prueba y se compara el resultado, buscando la transición de “cerrado” (closed, con respuesta) a “filtrado” (filtered, sin respuesta) en los casos DROP, o confirmando el ACCEPT explícito en los casos correspondientes.
- No interferencia: se genera tráfico en sentido contrario (o hacia el nodo no afectado por esa regla) para confirmar que el filtrado es específico del nodo de destino indicado con -d y no afecta al otro extremo de la topología.

Las herramientas usadas fueron: ping (ICMP), nmap -sT/-Pn (barrido TCP), Test-NetConnection de PowerShell y telnet (verificación de puertos desde Windows), nslookup (consultas DNS/UDP) y Wireshark para inspeccionar cada paquete, identificando IP origen/destino, protocolo, puerto y banderas TCP.

IV. Parte 1 – Reglas de tráfico hacia Windows (192.168.70.130)

Punto 1. Bloquear un puerto TCP específico

Objetivo: impedir el acceso al puerto TCP 8080 de Windows desde cualquier origen, sin afectar el resto del tráfico.

```
sudo iptables -A FORWARD -p tcp -d 192.168.70.130 --dport 8080 -j DROP
```

Antes de la regla

Desde Kali se generó tráfico de ping y conexiones TCP hacia Windows (puertos 443, 80 y 8080). El ping responde con normalidad y, para los tres puertos TCP, Windows responde con RST/RST-ACK: los puertos existen pero no tienen ningún servicio escuchando (estado “cerrado”, no filtrado).

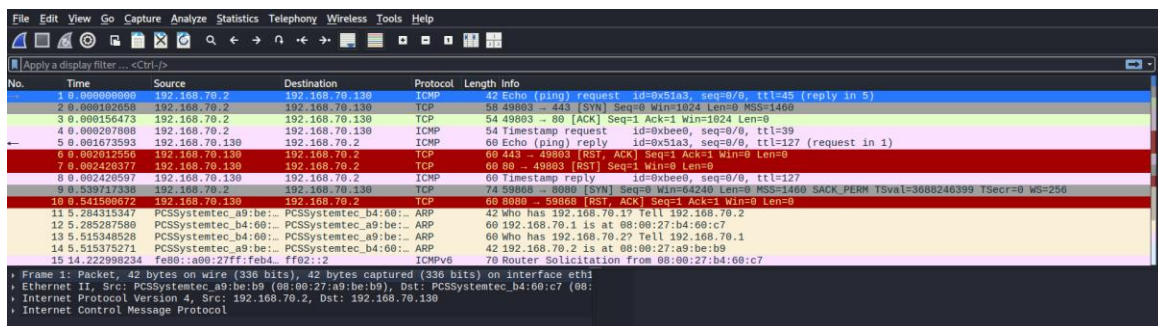


Fig. 2. Captura antes de la regla: ping exitoso y SYN a los puertos 443, 80 y 8080 respondidos con RST/RST-ACK (puertos cerrados, no bloqueados).

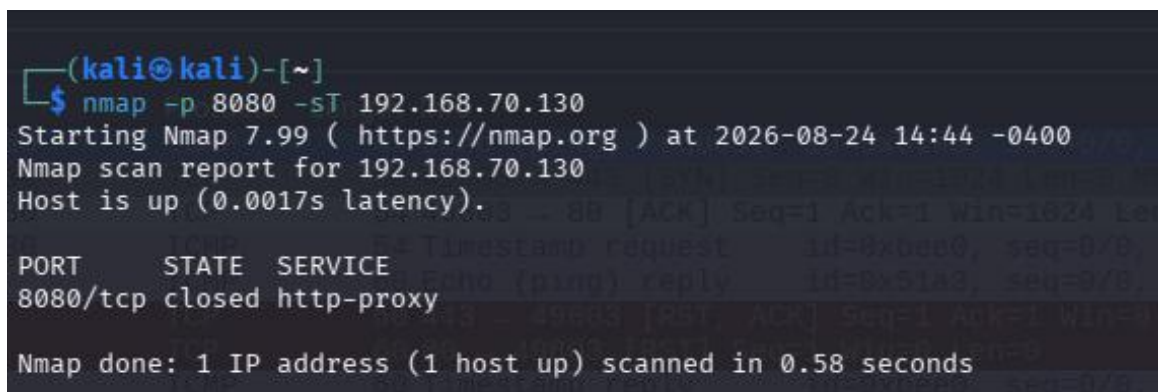


Fig. 3. nmap -p 8080 -sT confirma el estado closed antes de aplicar la regla.

Configuración de la regla

Se agrega la regla DROP para TCP/8080 con destino Windows. El contador de la regla en la cadena FORWARD queda en 0 hasta que llegue tráfico coincidente.

```
vboxuser@Ubuntu:~$ sudo iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in      out     source            destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in      out     source            destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in      out     source            destination
vboxuser@Ubuntu:~$ sudo iptables -A FORWARD -p tcp -d 192.168.70.130 --dport 8080 -j DROP
vboxuser@Ubuntu:~$ sudo iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in      out     source            destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in      out     source            destination
0      0 DROP        tcp  -- *      *       0.0.0.0/0         192.168.70.130    tcp dpt:8080

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in      out     source            destination
vboxuser@Ubuntu:~$
```

Fig. 4. Alta de la regla en FORWARD y verificación con iptables -L -n -v.

Después de la regla

El ping sigue funcionando con normalidad (la regla solo afecta TCP/8080). Los intentos de conexión a 443 y 80 se comportan igual que antes (RST, sin relación con esta regla), pero los dos intentos de SYN al puerto 8080 ya no reciben ninguna respuesta: el paquete se descarta en Ubuntu antes de llegar a Windows.

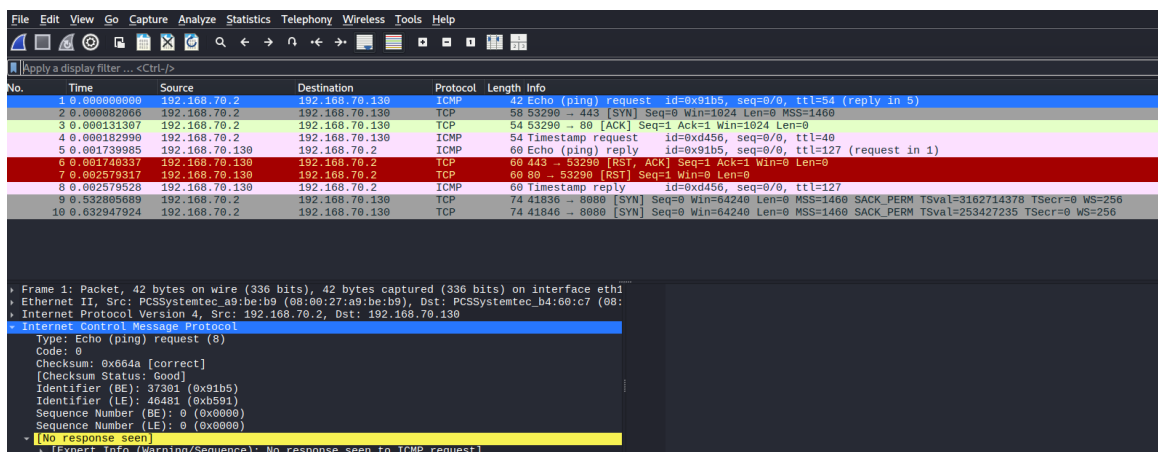


Fig. 5. Después de la regla: SYN al 8080 sin respuesta (filtered), mientras 443/80 siguen respondiendo con RST y el ping sigue vivo.

```
(kali@kali)-[~]
$ nmap -p 8080 -sT 192.168.70.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-24 14:52 -0400
Nmap scan report for 192.168.70.130
Host is up (0.0018s latency).
PORT      STATE      SERVICE
8080/tcp  filtered  http-proxy
Nmap done: 1 IP address (1 host up) scanned in 0.77 seconds
```

Fig. 6. nmap -p 8080 -sT ahora reporta filtered en lugar de closed, confirmando el bloqueo.

Prueba de que la regla no interfiere con el otro nodo

Desde Windows se probó el puerto 8080 de Kali con Test-NetConnection: el TCP falla porque Kali tampoco tiene nada escuchando ahí (comportamiento normal, no relacionado con la regla), pero el ping sí tiene éxito (PingSucceeded: True), confirmando que la regla —que solo aplica a -d 192.168.70.130— no afecta en absoluto al tráfico dirigido a Kali.

```
C:\Users\windows>powershell -Command "Test-NetConnection -ComputerName 192.168.70.2 -Port 8080"
ADVERTENCIA: TCP connect to (192.168.70.2 : 8080) failed

ComputerName      : 192.168.70.2
RemoteAddress     : 192.168.70.2
RemotePort        : 8080
InterfaceAlias    : Ethernet
SourceAddress     : 192.168.70.130
PingSucceeded     : True
PingReplyDetails  (RTT) : 1 ms
TcpTestSucceeded  : False
```

Fig. 7. Test-NetConnection desde Windows hacia Kali:8080 — el ping funciona con normalidad.

Punto 2. Bloquear tráfico ICMP (ping) hacia Windows

Objetivo: impedir que llegue tráfico ICMP con destino Windows.

```
sudo iptables -A FORWARD -p icmp -d 192.168.70.130 -j DROP
```

Antes de la regla

Un ping de 4 paquetes desde Kali hacia Windows se completa con 0% de pérdida, y Wireshark muestra los cuatro pares de Echo request/Echo reply completos.

```
(kali㉿kali)-[~]
$ ping -c 4 192.168.70.130
PING 192.168.70.130 (192.168.70.130) 56(84) bytes of data.
64 bytes from 192.168.70.130: icmp_seq=1 ttl=127 time=2.19 ms
64 bytes from 192.168.70.130: icmp_seq=2 ttl=127 time=1.70 ms
64 bytes from 192.168.70.130: icmp_seq=3 ttl=127 time=2.33 ms
64 bytes from 192.168.70.130: icmp_seq=4 ttl=127 time=2.69 ms

— 192.168.70.130 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 1.703/2.226/2.690/0.353 ms
```

Fig. 8. ping -c 4 desde Kali hacia Windows: 0% de pérdida.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.70.2	192.168.70.130	ICMP	98	Echo (ping) request id=0xe8f9, seq=1/256, ttl=64 (reply in 2)
2	0.002170199	192.168.70.130	192.168.70.2	ICMP	98	Echo (ping) reply id=0xe8f9, seq=1/256, ttl=127 (request in 1)
3	1.000583052	192.168.70.2	192.168.70.130	ICMP	98	Echo (ping) request id=0xe8f9, seq=2/512, ttl=64 (reply in 4)
4	1.002259474	192.168.70.130	192.168.70.2	ICMP	98	Echo (ping) reply id=0xe8f9, seq=2/512, ttl=127 (request in 3)
5	2.002451256	192.168.70.2	192.168.70.130	ICMP	98	Echo (ping) request id=0xe8f9, seq=3/768, ttl=64 (reply in 6)
6	2.004755709	192.168.70.130	192.168.70.2	ICMP	98	Echo (ping) reply id=0xe8f9, seq=3/768, ttl=127 (request in 5)
7	3.004177927	192.168.70.2	192.168.70.130	ICMP	98	Echo (ping) request id=0xe8f9, seq=4/1024, ttl=64 (reply in 8)
8	3.006047804	192.168.70.130	192.168.70.2	ICMP	98	Echo (ping) reply id=0xe8f9, seq=4/1024, ttl=127 (request in 7)
9	5.004374912	PCSSystemtec_b4:60::...	PCSSystemtec_a9:be::...	ARP	60	who has 192.168.70.2? Tell 192.168.70.1
10	5.004392963	PCSSystemtec_a9:be::...	PCSSystemtec_b4:60::...	ARP	42	192.168.70.2 is at 08:00:27:a9:be:b9
11	5.208621786	PCSSystemtec_a9:be::...	PCSSystemtec_b4:60::...	ARP	42	who has 192.168.70.1? Tell 192.168.70.2
12	5.209952012	PCSSystemtec_b4:60::...	PCSSystemtec_a9:be::...	ARP	60	192.168.70.1 is at 08:00:27:b4:60:c7

Fig. 9. Wireshark: los 4 pares Echo request/reply completos antes de la regla.

Configuración de la regla

```
vboxuser@Ubuntu:~$ sudo iptables -A FORWARD -p icmp -d 192.168.70.130 -j DROP
vboxuser@Ubuntu:~$ sudo iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source            destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source            destination
  0    0 DROP      tcp  --  *      *        0.0.0.0/0         192.168.70.130    tcp dpt:8080
  0    0 DROP      icmp --  *      *        0.0.0.0/0         192.168.70.130

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source            destination
```

Fig. 10. Regla DROP para ICMP agregada en FORWARD (además de la regla TCP/8080 del punto anterior).

Después de la regla

Al repetir el mismo ping desde Kali, Wireshark solo registra las cuatro solicitudes salientes marcadas “(no response found!)”: Ubuntu descarta cada Echo request antes de reenviarlo a la red de Windows. En la terminal de Kali se ve el contraste directo entre el ping anterior (0% de pérdida) y este mismo ping después de la regla (100% de pérdida).

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.70.2	192.168.70.130	ICMP	98	Echo (ping) request id=0xe8fa, seq=1/256, ttl=64 (no response found!)
2	1.010097000	192.168.70.2	192.168.70.130	ICMP	98	Echo (ping) request id=0xe8fa, seq=2/512, ttl=64 (no response found!)
3	2.034510191	192.168.70.2	192.168.70.130	ICMP	98	Echo (ping) request id=0xe8fa, seq=3/768, ttl=64 (no response found!)
4	3.057757212	192.168.70.2	192.168.70.130	ICMP	98	Echo (ping) request id=0xe8fa, seq=4/1024, ttl=64 (no response found!)
5	5.137622298	PCSSystemtec_a9:be:...	PCSSystemtec_b4:60:...	ARP	42	who has 192.168.70.1? Tell 192.168.70.2
6	5.138985256	PCSSystemtec_b4:60:...	PCSSystemtec_a9:be:...	ARP	60	192.168.70.1 is at 08:00:27:b4:60:c7

Frame 1: Packet, 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface eth1, id 0 Ethernet II, Src: PCSSystemtec_a9:be:b9 (08:00:27:a9:be:b9), Dst: PCSSystemtec_b4:60:c7 (08:00:27:b4:60:c7) Internet Protocol Version 4, Src: 192.168.70.2, Dst: 192.168.70.130 Internet Control Message Protocol Type: Echo (ping) request (8) Code: 0 Checksum: 0x4f3e [correct] [Checksum Status: Good] Identifier (BE): 59642 (0xe8fa) Identifier (LE): 64232 (0xfae8) Sequence Number (BE): 1 (0x0001) Sequence Number (LE): 256 (0x0100) [No response seen] [Expert Info (Warning/Sequence): No response seen to ICMP request] ICMP Data: 6790936a00000000f8f70d000000000101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d2e2f3031323334353637

Fig. 11. Después de la regla: las 4 solicitudes ICMP hacia Windows quedan sin respuesta.


```
(kali㉿kali)-[~]
$ ping -c 4 192.168.70.130
PING 192.168.70.130 (192.168.70.130) 56(84) bytes of data.
64 bytes from 192.168.70.130: icmp_seq=1 ttl=127 time=2.19 ms
64 bytes from 192.168.70.130: icmp_seq=2 ttl=127 time=1.70 ms
64 bytes from 192.168.70.130: icmp_seq=3 ttl=127 time=2.33 ms
64 bytes from 192.168.70.130: icmp_seq=4 ttl=127 time=2.69 ms

— 192.168.70.130 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 1.703/2.226/2.690/0.353 ms

(kali㉿kali)-[~]
$ ping -c 4 192.168.70.130
PING 192.168.70.130 (192.168.70.130) 56(84) bytes of data.

— 192.168.70.130 ping statistics —
4 packets transmitted, 0 received, 100% packet loss, time 3058ms
```

Fig. 12. Comparación en la terminal de Kali: 0% de pérdida antes vs. 100% de pérdida después de agregar la regla.

Prueba de no interferencia (y un hallazgo importante)

Se probó el sentido contrario: ping iniciado desde Windows hacia Kali. La captura de Wireshark, tomada sobre el segmento Kali–Ubuntu, muestra el intercambio completo de Echo request/reply, porque Kali sí recibe la solicitud y sí genera la respuesta de inmediato. Sin embargo, en la consola de Windows el resultado fue 100% de paquetes perdidos.

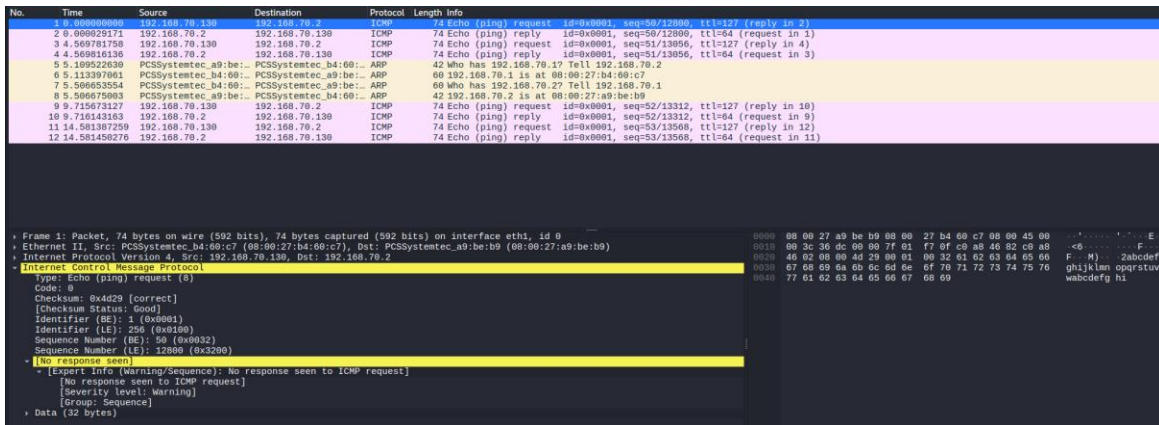


Fig. 13. Wireshark (segmento Kali–Ubuntu): el ping Windows→Kali parece completo — Kali responde a cada solicitud.

```
C:\Users\windows>ping 192.168.70.2

Haciendo ping a 192.168.70.2 con 32 bytes de datos:
Tiempo de espera agotado para esta solicitud.
Tiempo de espera agotado para esta solicitud.
Tiempo de espera agotado para esta solicitud.
Tiempo de espera agotado para esta solicitud.

Estadísticas de ping para 192.168.70.2:
    Paquetes: enviados = 4, recibidos = 0, perdidos = 4
      (100% perdidos),

C:\Users\windows>
```

Fig. 14. Sin embargo, en Windows el mismo ping reporta 100% de paquetes perdidos.

La explicación es que la regla filtra por dirección IP de destino, sin distinguir si el paquete ICMP es una solicitud o una respuesta: la respuesta que genera Kali también tiene como destino 192.168.70.130, así que al intentar cruzar de vuelta hacia la red de Windows, Ubuntu la descarta igual que descartaría una solicitud. Por eso Kali “ve” el ida y vuelta completo en su propio segmento, pero esa respuesta nunca llega a Windows. Esto demuestra que una regla DROP basada únicamente en -d bloquea todo el tráfico ICMP dirigido a esa IP, sin importar si la conversación fue iniciada por el otro extremo; para permitir que las respuestas a pings iniciados por Windows regresen sin abrir también el paso a pings iniciados por Kali, sería necesario un filtrado con estado (por ejemplo, -m state --state ESTABLISHED,RELATED -j ACCEPT) o distinguir por tipo de mensaje ICMP (echo-request vs. echo-reply) en lugar de bloquear por IP de destino en bloque.

Punto 3. Permitir únicamente tráfico TCP hacia un puerto específico

Objetivo: dejar accesible solo el puerto TCP 22 de Windows y bloquear cualquier otro puerto TCP.

```
sudo iptables -A FORWARD -p tcp -d 192.168.70.130 --dport 22 -j ACCEPT
sudo iptables -A FORWARD -p tcp -d 192.168.70.130 -j DROP
```

Antes de la regla

El ping ya no responde (regla del punto 2, aún activa). Los puertos 443, 80 y 22 responden todos con RST/RST-ACK: cerrados, pero alcanzables.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.70.2	192.168.70.130	ICMP	42	Echo (ping) request id=0xb6eb, seq=0/0, ttl=40 (no response found!)
2	0.000408006	192.168.70.2	192.168.70.130	TCP	58	56411 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
3	0.000898075	192.168.70.2	192.168.70.130	TCP	54	56411 → 80 [ACK] Seq=1 Ack=1 Win=1024 Len=0
4	0.001291776	192.168.70.2	192.168.70.130	ICMP	54	Timestamp request id=0xd076, seq=0/0, ttl=47
5	0.005720502	192.168.70.130	192.168.70.2	TCP	60	443 → 56411 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
6	0.007902174	192.168.70.130	192.168.70.2	TCP	60	80 → 56411 [RST] Seq=1 Win=0 Len=0
7	0.020888307	192.168.70.2	192.168.70.130	TCP	74	59364 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=804395255 TSecr=0 WS=256
8	0.032755175	192.168.70.130	192.168.70.2	TCP	60	22 → 59364 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
9	0.034601779	PCSSystemtec_b4:60::	PCSSystemtec_a9:be::	ARP	60	Who has 192.168.70.2? Tell 192.168.70.1
10	0.034606480	PCSSystemtec_a9:be::	PCSSystemtec_b4:60::	ARP	42	192.168.70.2 is at 08:00:27:a9:be:b9
11	0.170660734	192.168.70.2	192.168.70.130	ICMP	42	Echo (ping) request id=0xb05d, seq=0/0, ttl=57 (no response found!)
12	0.170579763	192.168.70.2	192.168.70.130	TCP	58	62482 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
13	0.170693797	192.168.70.2	192.168.70.130	TCP	54	62482 → 80 [ACK] Seq=1 Ack=1 Win=1024 Len=0
14	0.170790783	192.168.70.2	192.168.70.130	ICMP	54	Timestamp request id=0xc096, seq=0/0, ttl=59
15	0.182066153	192.168.70.130	192.168.70.2	TCP	60	443 → 62482 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
16	0.182069178	192.168.70.130	192.168.70.2	TCP	60	80 → 62482 [RST] Seq=1 Win=0 Len=0
17	0.1707573843	192.168.70.2	192.168.70.130	TCP	74	47100 → 8081 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2095638886 TSecr=0 WS=256
18	0.1717679286	192.168.70.130	192.168.70.2	TCP	60	8081 → 47100 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

* Frame 1: Packet, 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface eth1, id 0
 * Ethernet II, Src: PCSSystemtec_a9:be:b9 (08:00:27:a9:be:b9), Dst: PCSSystemtec_b4:60:c7 (08:00:27:b4:60:c7)
 * Internet Protocol Version 4, Src: 192.168.70.2, Dst: 192.168.70.130
 * Internet Control Message Protocol
 Type: Echo (ping) request (8)
 Code: 0
 Checksum: 0x9714 [correct]
 [Checksum Status: Good]
 Identifier (BE): 24811 (0x60eb)
 Identifier (LE): 60256 (0xeb00)
 Sequence Number (BE): 0 (0x0000)
 Sequence Number (LE): 0 (0x0000)
 * [No response seen]
 * [Expert Info (Warning/Sequence): No response seen to ICMP request]
 [No response seen to ICMP request]
 [Severity level: Warning]
 [Group: Sequence]

Fig. 15. Antes de la regla: 443, 80 y 22 responden con RST (cerrados, sin bloqueo).

```

(kali@kali)-[~]
$ nmap -p 22 -sT 192.168.70.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-29 22:56 -0400
Nmap scan report for 192.168.70.130
Host is up (0.0051s latency).

PORT      STATE SERVICE
22/tcp    closed ssh

Nmap done: 1 IP address (1 host up) scanned in 0.60 seconds

(kali@kali)-[~]
$ nmap -p 8081 -sT 192.168.70.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-29 22:56 -0400
Nmap scan report for 192.168.70.130
Host is up (0.0061s latency).

PORT      STATE SERVICE
8081/tcp  closed blackice-icecap

Nmap done: 1 IP address (1 host up) scanned in 0.61 seconds

```

Fig. 16. nmap confirma 22 y 8081 en estado closed.

Configuración de la regla

Se agrega primero el ACCEPT específico para el puerto 22 y luego el DROP genérico para el resto de TCP; el orden importa, porque iptables evalúa las reglas en secuencia y aplica la primera que coincide.

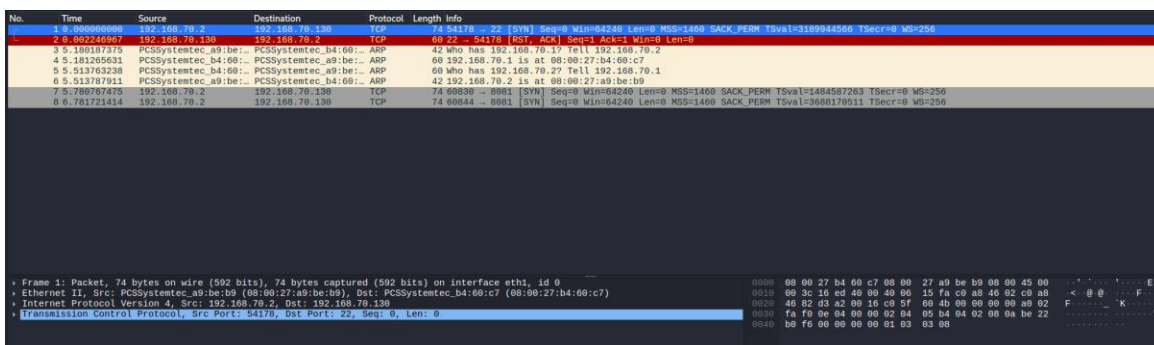
```
vboxuser@Ubuntu:~$ sudo iptables -A FORWARD -p tcp -d 192.168.70.130 --dport 22 -j ACCEPT
vboxuser@Ubuntu:~$ sudo iptables -A FORWARD -p tcp -d 192.168.70.130 -j DROP
vboxuser@Ubuntu:~$ sudo iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in      out     source            destination

Chain FORWARD (policy ACCEPT 72 packets, 3724 bytes)
pkts bytes target      prot opt in      out     source            destination
 24 1128 DROP        icmp -- *       *        0.0.0.0/0         192.168.70.130
  0    0 DROP        tcp  -- *       *        0.0.0.0/0         192.168.70.130      tcp dpt:8080
  0    0 ACCEPT      tcp  -- *       *        0.0.0.0/0         192.168.70.130      tcp dpt:22
  0    0 DROP        tcp  -- *       *        0.0.0.0/0         192.168.70.130
```

Fig. 17. Reglas ACCEPT (tcp/22) y DROP (resto de tcp) agregadas a FORWARD, sobre las reglas previas de los puntos 1 y 2.

Después de la regla

El puerto 22 sigue respondiendo con RST/ACK: el ACCEPT deja pasar el paquete y es Windows —que no tiene ningún servicio SSH activo— quien lo rechaza. El puerto 8081, en cambio, ya no recibe ninguna respuesta: cae en la regla DROP genérica.



The image shows a Wireshark packet capture. The first packet is a SYN packet from 192.168.70.2 to 192.168.70.130 on port 22. The second packet is a RST/ACK packet from 192.168.70.130 to 192.168.70.2 on port 22. The third packet is a SYN packet from 192.168.70.2 to 192.168.70.130 on port 8081, which is marked as filtered. The packet details show the SYN packet to port 22 has a sequence number of 64240 and a window size of 64240. The RST/ACK packet has a sequence number of 64240 and a window size of 64240. The filtered SYN packet to port 8081 has a sequence number of 64240 and a window size of 64240.

Fig. 18. Después de la regla: SYN al 22 responde RST/ACK (ilega, ACCEPT); SYN al 8081 sin respuesta (filtered, DROP).

```
(kali@kali)-[~]
$ nmap -p 22 -sT -Pn 192.168.70.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-29 23:09 -0400
Nmap scan report for 192.168.70.130
Host is up (0.0021s latency).

PORT      STATE SERVICE
22/tcp    closed ssh

Nmap done: 1 IP address (1 host up) scanned in 0.53 seconds

(kali@kali)-[~]
$ nmap -p 8081 -sT -Pn 192.168.70.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-29 23:09 -0400
Nmap scan report for 192.168.70.130
Host is up.

PORT      STATE SERVICE
8081/tcp  filtered blackice-icecap

Nmap done: 1 IP address (1 host up) scanned in 2.53 seconds
```

Fig. 19. nmap -Pn confirma: 22 = closed (ilega hasta el sistema) y 8081 = filtered (bloqueado en el camino).

Prueba de no interferencia

Desde Windows se probaron los puertos 22 y 8081 de Kali (que no tiene ninguna regla propia todavía) mediante escaneo y telnet: ambos responden como cerrados normales, sin relación con las reglas orientadas a Windows.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.00000000	192.168.70.1	224.0.0.251	MDNS	87	Standard query 0x0000 PTR _ipps._tcp.local, "QM" question PTR _ipps._tcp.local, "QM" question
2	0.281191380	fe80::a00:27ff:feb4::ff02::fb	ff02::fb	MDNS	107	Standard query 0x0000 PTR _ipps._tcp.local, "QM" question PTR _ipps._tcp.local, "QM" question
3	172.232833955	fe80::a00:27ff:feb4::ff02::fb	ff02::fb	ICMPv6	70	Router Solicitation from 08:00:27:b4:60:c7
4	478.442898183	192.168.70.130	192.168.70.2	TCP	66	60860 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
5	478.442135867	192.168.70.2	192.168.70.130	TCP	54	22 → 60860 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
6	479.463511982	192.168.70.130	192.168.70.2	TCP	66	[TCP Port numbers reused] 60860 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
7	479.463536800	192.168.70.2	192.168.70.130	TCP	54	22 → 60860 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
8	481.480852097	192.168.70.130	192.168.70.2	TCP	66	[TCP Port numbers reused] 60860 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
9	481.480877539	192.168.70.2	192.168.70.130	TCP	54	22 → 60860 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
10	483.526388118	PCSSystemtec_b4:60::	PCSSystemtec_a9:be::	ARP	60	Who has 192.168.70.2? Tell 192.168.70.1
11	483.526324915	PCSSystemtec_a9:be::	PCSSystemtec_b4:60::	ARP	42	192.168.70.2 is at 08:00:27:a9:be:b9
12	483.675973483	PCSSystemtec_a9:be::	PCSSystemtec_b4:60::	ARP	42	Who has 192.168.70.1? Tell 192.168.70.2
13	483.676870900	PCSSystemtec_b4:60::	PCSSystemtec_a9:be::	ARP	60	192.168.70.1 is at 08:00:27:b4:60:c7
14	485.493467613	192.168.70.130	192.168.70.2	TCP	66	[TCP Port numbers reused] 60860 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
15	485.493505783	192.168.70.2	192.168.70.130	TCP	54	22 → 60860 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
16	493.517468330	192.168.70.130	192.168.70.2	TCP	66	[TCP Port numbers reused] 60860 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
17	493.517487115	192.168.70.2	192.168.70.130	TCP	54	22 → 60860 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
18	518.608569687	192.168.70.130	192.168.70.2	TCP	66	60861 → 8081 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
19	518.608595507	192.168.70.2	192.168.70.130	TCP	54	8081 → 60861 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
20	519.618080007	192.168.70.130	192.168.70.2	TCP	66	[TCP Port numbers reused] 60861 → 8081 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
21	519.618100990	192.168.70.2	192.168.70.130	TCP	54	8081 → 60861 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
22	521.618120951	192.168.70.130	192.168.70.2	TCP	66	[TCP Port numbers reused] 60861 → 8081 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
23	521.618143801	192.168.70.2	192.168.70.130	TCP	54	8081 → 60861 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
24	523.612665200	PCSSystemtec_a9:be::	PCSSystemtec_b4:60::	ARP	42	Who has 192.168.70.1? Tell 192.168.70.2
25	523.613136952	PCSSystemtec_b4:60::	PCSSystemtec_a9:be::	ARP	60	192.168.70.1 is at 08:00:27:b4:60:c7
26	523.973738159	PCSSystemtec_b4:60::	PCSSystemtec_a9:be::	ARP	60	Who has 192.168.70.2? Tell 192.168.70.1
27	523.973747587	PCSSystemtec_a9:be::	PCSSystemtec_b4:60::	ARP	42	192.168.70.2 is at 08:00:27:a9:be:b9
28	523.973750995	192.168.70.130	192.168.70.2	TCP	66	[TCP Port numbers reused] 60861 → 8081 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
29	525.621402647	192.168.70.2	192.168.70.130	TCP	54	8081 → 60861 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
30	533.634835936	192.168.70.130	192.168.70.2	TCP	66	[TCP Port numbers reused] 60861 → 8081 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
31	533.634866671	192.168.70.2	192.168.70.130	TCP	54	8081 → 60861 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Fig. 20. Escaneo Windows→Kali a los puertos 22 y 8081: ambos cerrados de forma normal (RST), sin bloqueo.

```
C:\Windows\system32>telnet 192.168.70.2 22
Conectándose a 192.168.70.2...No se puede abrir la conexión al host, en puerto 22: Error en la conexión

C:\Windows\system32>telnet 192.168.70.2 8081
Conectándose a 192.168.70.2...No se puede abrir la conexión al host, en puerto 8081: Error en la conexión
```

Fig. 21. telnet desde Windows hacia Kali:22 y Kali:8081 — conexión rechazada de forma normal.

V. Parte 2 – Reglas de tráfico hacia Kali (192.168.70.2)

Punto 4. Bloquear un puerto TCP específico

Objetivo: impedir el acceso al puerto TCP 21 de Kali.

```
sudo iptables -A FORWARD -p tcp -d 192.168.70.2 --dport 21 -j DROP
```

Antes de la regla

Windows intenta repetidamente conectarse al puerto 21 de Kali; Kali responde de inmediato con RST/ACK (puerto cerrado, sin firewall de por medio). El ping Windows→Kali que aparece en la misma captura ya se ve afectado por la regla del punto 2 (que sigue activa desde la Parte 1): Test-NetConnection reporta el ping como fallido, aunque no es algo relacionado con esta regla.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.70.130	192.168.70.2	TCP	60	60863 → 21 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2	0.000037297	192.168.70.2	192.168.70.130	TCP	54	21 → 60863 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
3	1.006089609	192.168.70.130	192.168.70.2	TCP	60	[TCP Port numbers reused] 60863 → 21 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
4	1.006117534	192.168.70.2	192.168.70.130	TCP	54	21 → 60863 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
5	3.009046570	192.168.70.130	192.168.70.2	TCP	60	[TCP Port numbers reused] 60863 → 21 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
6	3.009116897	192.168.70.2	192.168.70.130	TCP	54	21 → 60863 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
7	5.109769738	PCSSystemtec_a9:be:...	PCSSystemtec_b4:60:...	ARP	42	Who has 192.168.70.1? Tell 192.168.70.2
8	5.110768593	PCSSystemtec_b4:60:...	PCSSystemtec_a9:be:...	ARP	60	192.168.70.1 is at 08:00:27:b4:60:c7
9	5.494102851	PCSSystemtec_b4:60:...	PCSSystemtec_a9:be:...	ARP	60	Who has 192.168.70.2? Tell 192.168.70.1
10	5.494117412	PCSSystemtec_a9:be:...	PCSSystemtec_b4:60:...	ARP	42	192.168.70.2 is at 08:00:27:a9:be:b9
11	7.031519837	192.168.70.130	192.168.70.2	TCP	60	[TCP Port numbers reused] 60863 → 21 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
12	7.031548644	192.168.70.2	192.168.70.130	TCP	54	21 → 60863 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
13	15.054080250	192.168.70.130	192.168.70.2	TCP	60	[TCP Port numbers reused] 60863 → 21 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
14	15.054081132	192.168.70.2	192.168.70.130	TCP	54	21 → 60863 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
15	21.217574341	192.168.70.130	192.168.70.2	ICMP	74	Echo (ping) request id=0x0001, seq=55/14080, ttl=127 (reply in 16)
16	21.217599376	192.168.70.2	192.168.70.130	ICMP	74	Echo (ping) reply id=0x0001, seq=55/14080, ttl=64 (request in 15)

Fig. 22. Antes de la regla: SYN al puerto 21 de Kali respondido con RST/ACK.

```
C:\Windows\system32>powershell Test-Netconnection -ComputerName 192.168.70.2 -Port 21
ADVERTENCIA: TCP connect to (192.168.70.2 : 21) failed
ADVERTENCIA: Ping to 192.168.70.2 failed with status: TimedOut

ComputerName       : 192.168.70.2
RemoteAddress      : 192.168.70.2
RemotePort         : 21
InterfaceAlias     : Ethernet
SourceAddress      : 192.168.70.130
PingSucceeded      : False
PingReplyDetails (RTT) : 0 ms
TcpTestSucceeded   : False
```

Fig. 23. Test-NetConnection: TCP al 21 falla (cerrado) y el ping muestra TimedOut — efecto arrastrado de la regla del punto 2.

Configuración de la regla

```
vboxuser@Ubuntu:~$ sudo iptables -A FORWARD -p tcp -d 192.168.70.2 --dport 21 -j DROP
[sudo: authenticate] Password:
vboxuser@Ubuntu:~$ sudo iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain FORWARD (policy ACCEPT 96 packets, 4964 bytes)
 pkts bytes target     prot opt in     out     source               destination
 38   1656 DROP       icmp -- *       *       0.0.0.0/0            192.168.70.130
 0      0 DROP       tcp  -- *       *       0.0.0.0/0            192.168.70.130    tcp dpt:8080
 2     120 ACCEPT     tcp  -- *       *       0.0.0.0/0            192.168.70.130    tcp dpt:22
 36   1544 DROP       tcp  -- *       *       0.0.0.0/0            192.168.70.130
 0      0 DROP       tcp  -- *       *       0.0.0.0/0            192.168.70.2      tcp dpt:21

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
```

Fig. 24. Nueva regla DROP tcp/21 hacia Kali, sumada a las reglas ya vigentes de la Parte 1.

Después de la regla

El ping Windows→Kali (no relacionado con esta regla) se documentó de nuevo, mostrando el mismo patrón visto en el punto 2: un intercambio ICMP que aparenta completarse en el segmento de Kali. En cuanto al puerto 21, Test-NetConnection reporta el mismo mensaje de fallo que antes de aplicar la regla (“TCP connect ... failed”); el texto por sí solo no distingue si el puerto está cerrado (RST inmediato) o bloqueado (silencio total). Esa diferencia solo se puede confirmar con Wireshark o con nmap -Pn, tal como se hizo en los puntos 1 y 3: antes de la regla se observan paquetes RST/ACK de vuelta, y después de la regla los SYN quedan sin ninguna respuesta.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.70.130	192.168.70.2	ICMP	74	Echo (ping) request id=0x0001, seq=56/14336, ttl=127 (reply in 2)
2	0.000027835	192.168.70.2	192.168.70.130	ICMP	74	Echo (ping) reply id=0x0001, seq=56/14336, ttl=64 (request in 1)
3	5.044694720	PCSSystemtec_a9:be:...	PCSSystemtec_b4:60:...	ARP	42	Who has 192.168.70.1? Tell 192.168.70.2
4	5.045756889	PCSSystemtec_b4:60:...	PCSSystemtec_a9:be:...	ARP	60	192.168.70.1 is at 08:00:27:b4:60:c7
5	5.416338364	PCSSystemtec_b4:60:...	PCSSystemtec_a9:be:...	ARP	60	Who has 192.168.70.2? Tell 192.168.70.1
6	5.416364778	PCSSystemtec_a9:be:...	PCSSystemtec_b4:60:...	ARP	42	192.168.70.2 is at 08:00:27:a9:be:b9

Fig. 25. Ping Windows→Kali tras agregar la regla del puerto 21 (comportamiento ICMP igual al del punto 2, no relacionado con esta regla).

```
C:\Windows\system32>powershell Test-Netconnection -ComputerName 192.168.70.2 -Port 21
ADVERTENCIA: TCP connect to (192.168.70.2 : 21) failed
ADVERTENCIA: Ping to 192.168.70.2 failed with status: TimedOut

ComputerName           : 192.168.70.2
RemoteAddress          : 192.168.70.2
RemotePort             : 21
InterfaceAlias         : Ethernet
SourceAddress          : 192.168.70.130
PingSucceeded          : False
PingReplyDetails (RTT) : 0 ms
TcpTestSucceeded       : False
```

Fig. 26. Test-NetConnection al puerto 21 después de la regla: el mensaje de fallo es igual al de antes; la diferencia real (RST vs. silencio) se confirma con Wireshark/nmap, no con este texto.

Punto 5. Bloquear tráfico UDP hacia un puerto específico

Objetivo: impedir el acceso al puerto UDP 53 (DNS) de Kali.

```
sudo iptables -A FORWARD -p udp -d 192.168.70.2 --dport 53 -j DROP
```

Antes de la regla

Windows envía consultas DNS (PTR, A, AAAA) a Kali, que no tiene ningún servicio DNS activo. Kali responde a cada consulta con un ICMP “Destination Unreachable (Port Unreachable)”: el puerto está cerrado, pero el host es alcanzable y responde. Desde el punto de vista de nslookup en Windows, el resultado visible es igualmente un timeout, porque nunca llega una respuesta DNS válida, aunque sí llegó una respuesta ICMP informativa.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.70.130	192.168.70.2	DNS	85	Standard query 0x0001 PTR 2.70.168.192.in-addr.arpa
2	0.000087386	192.168.70.2	192.168.70.130	ICMP	113	Destination unreachable (Port unreachable)
3	2.012651121	192.168.70.130	192.168.70.2	DNS	70	Standard query 0x0002 A google.com
4	2.012680361	192.168.70.2	192.168.70.130	ICMP	98	Destination unreachable (Port unreachable)
5	4.012191359	192.168.70.130	192.168.70.2	DNS	70	Standard query 0x0003 AAAA google.com
6	4.012225233	192.168.70.2	192.168.70.130	ICMP	98	Destination unreachable (Port unreachable)
7	5.074101660	PCSSystemtec_b4:60:...	PCSSystemtec_a9:be:...	ARP	60	Who has 192.168.70.2? Tell 192.168.70.1
8	5.074124114	PCSSystemtec_a9:be:...	PCSSystemtec_b4:60:...	ARP	42	192.168.70.2 is at 08:00:27:a9:be:b9
9	5.249121770	PCSSystemtec_a9:be:...	PCSSystemtec_b4:60:...	ARP	42	Who has 192.168.70.1? Tell 192.168.70.2
10	5.250638704	PCSSystemtec_b4:60:...	PCSSystemtec_a9:be:...	ARP	60	192.168.70.1 is at 08:00:27:b4:60:c7
11	6.029409526	192.168.70.130	192.168.70.2	DNS	70	Standard query 0x0004 A google.com
12	6.029433868	192.168.70.2	192.168.70.130	ICMP	98	Destination unreachable (Port unreachable)
13	8.041027973	192.168.70.130	192.168.70.2	DNS	70	Standard query 0x0005 AAAA google.com
14	8.041071631	192.168.70.2	192.168.70.130	ICMP	98	Destination unreachable (Port unreachable)

Fig. 27. Antes de la regla: cada consulta DNS recibe un ICMP Port Unreachable de Kali (puerto cerrado, host alcanzable).


```

C:\Windows\system32>nslookup google.com 192.168.70.2
DNS request timed out.
    timeout was 2 seconds.
Servidor: UnKnown
Address: 192.168.70.2

DNS request timed out.
    timeout was 2 seconds.
DNS request timed out.
    timeout was 2 seconds.
DNS request timed out.
    timeout was 2 seconds.
DNS request timed out.
    timeout was 2 seconds.
*** Se agotó el tiempo de espera de la solicitud a UnKnown

```

Fig. 28. nslookup en Windows: timeout, aunque Kali sí respondió a nivel ICMP.

Configuración de la regla

```

vboxuser@Ubuntu:~$ sudo iptables -A FORWARD -p udp -d 192.168.70.2 --dport 53 -j DROP
vboxuser@Ubuntu:~$ sudo iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination

Chain FORWARD (policy ACCEPT 107 packets, 5614 bytes)
 pkts bytes target    prot opt in     out     source                   destination
 49  2586 DROP      icmp -- *      *       0.0.0.0/0               192.168.70.130
  0     0 DROP      tcp  -- *      *       0.0.0.0/0               192.168.70.130    tcp dpt:8080
  2    120 ACCEPT    tcp  -- *      *       0.0.0.0/0               192.168.70.130    tcp dpt:22
 36   1544 DROP      tcp  -- *      *       0.0.0.0/0               192.168.70.130
  5    260 DROP      tcp  -- *      *       0.0.0.0/0               192.168.70.2      tcp dpt:21
  0     0 DROP      udp  -- *      *       0.0.0.0/0               192.168.70.2      udp dpt:53

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination

```

Fig. 29. Regla DROP udp/53 hacia Kali agregada al conjunto acumulado de reglas.

Después de la regla

El resultado visible en nslookup es el mismo mensaje de timeout que antes, pero la causa cambió por completo. Para comprobarlo se capturó simultáneamente en dos puntos:


```
C:\Windows\system32>nslookup google.com 192.168.70.2
DNS request timed out.
    timeout was 2 seconds.
Servidor: UnKnown
Address: 192.168.70.2

DNS request timed out.
    timeout was 2 seconds.
DNS request timed out.
    timeout was 2 seconds.
DNS request timed out.
    timeout was 2 seconds.
DNS request timed out.
    timeout was 2 seconds.
*** Se agotó el tiempo de espera de la solicitud a UnKnown
```

Fig. 30. nslookup después de la regla: mismo mensaje de timeout en apariencia.

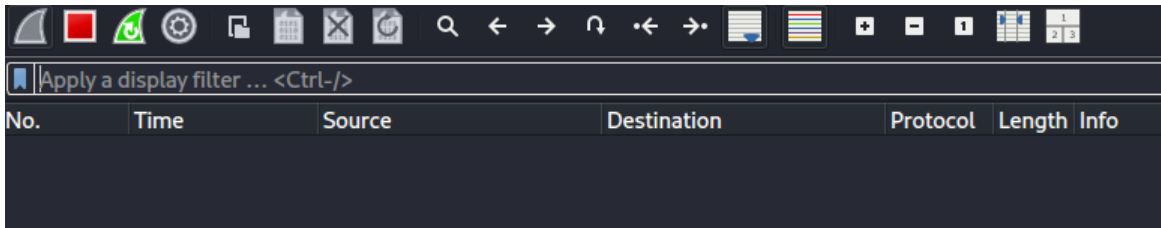


Fig. 31. Captura en la propia interfaz de Kali: no llega absolutamente ningún paquete — Ubuntu lo descartó antes de reenviarlo.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.70.130	192.168.70.2	DNS	85	Standard query 0x0001 PTR 2.70.168.192.in-addr.arpa
2	2.007179735	192.168.70.130	192.168.70.2	DNS	70	Standard query 0x0002 A google.com
3	4.023065931	192.168.70.130	192.168.70.2	DNS	70	Standard query 0x0003 AAAA google.com
4	6.052396554	192.168.70.130	192.168.70.2	DNS	70	Standard query 0x0004 A google.com
5	8.052831841	192.168.70.130	192.168.70.2	DNS	70	Standard query 0x0005 AAAA google.com

Fig. 32. Captura en Ubuntu, del lado de la red Ubuntu–Windows: las 5 consultas DNS sí salen de Windows y llegan hasta Ubuntu.

La comparación de ambas capturas confirma exactamente dónde ocurre el bloqueo: el tráfico sale de Windows y llega a Ubuntu con normalidad, pero Ubuntu lo descarta en su cadena FORWARD antes de reenviarlo hacia Kali, que nunca llega a ver esas consultas. Esta es la diferencia clave frente al estado “cerrado” de antes de la regla: ya no hay ningún tipo de respuesta (ni siquiera un Port Unreachable), porque el paquete ni siquiera llega a su destino.

Punto 6. Permitir tráfico ICMP hacia Kali

Objetivo: agregar una regla explícita que permita el tráfico ICMP con destino Kali.

```
sudo iptables -A FORWARD -p icmp -d 192.168.70.2 -j ACCEPT
```

Antes de la regla

Se probó el ping Windows→Kali: en el segmento de Kali, Wireshark vuelve a mostrar el intercambio completo de Echo request/reply (igual que en los puntos 2 y 4), pero en la consola de Windows el resultado es 100% de paquetes perdidos, por el mismo motivo ya explicado: la respuesta de Kali, destinada a 192.168.70.130, es descartada por la regla del punto 2 al intentar cruzar hacia la red de Windows.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.70.130	192.168.70.2	ICMP	74	Echo (ping) request id=0x0001, seq=57/14592, ttl=127 (reply in 2)
2	0.000054397	192.168.70.2	192.168.70.130	ICMP	74	Echo (ping) reply id=0x0001, seq=57/14592, ttl=64 (request in 1)
3	4.606186948	192.168.70.130	192.168.70.2	ICMP	74	Echo (ping) request id=0x0001, seq=58/14848, ttl=127 (reply in 4)
4	4.606217421	192.168.70.2	192.168.70.130	ICMP	74	Echo (ping) reply id=0x0001, seq=58/14848, ttl=64 (request in 3)
5	5.226144080	PCSSystemtec_a9:be:...	PCSSystemtec_b4:60:...	ARP	42	Who has 192.168.70.1? Tell 192.168.70.2
6	5.227187462	PCSSystemtec_b4:60:...	PCSSystemtec_a9:be:...	ARP	60	192.168.70.1 is at 08:00:27:b4:60:c7
7	5.295798704	PCSSystemtec_b4:60:...	PCSSystemtec_a9:be:...	ARP	60	Who has 192.168.70.2? Tell 192.168.70.1
8	5.295812368	PCSSystemtec_a9:be:...	PCSSystemtec_b4:60:...	ARP	42	192.168.70.2 is at 08:00:27:a9:be:b9
9	9.573762729	192.168.70.130	192.168.70.2	ICMP	74	Echo (ping) request id=0x0001, seq=59/15104, ttl=127 (reply in 10)
10	9.573787280	192.168.70.2	192.168.70.130	ICMP	74	Echo (ping) reply id=0x0001, seq=59/15104, ttl=64 (request in 9)
11	14.573368281	192.168.70.130	192.168.70.2	ICMP	74	Echo (ping) request id=0x0001, seq=60/15360, ttl=127 (reply in 12)
12	14.573392942	192.168.70.2	192.168.70.130	ICMP	74	Echo (ping) reply id=0x0001, seq=60/15360, ttl=64 (request in 11)

Fig. 33. Antes de la regla: el ping Windows→Kali se ve completo desde el segmento de Kali...

```
C:\Windows\system32>ping 192.168.70.2

Haciendo ping a 192.168.70.2 con 32 bytes de datos:
Tiempo de espera agotado para esta solicitud.
Tiempo de espera agotado para esta solicitud.
Tiempo de espera agotado para esta solicitud.
Tiempo de espera agotado para esta solicitud.

Estadísticas de ping para 192.168.70.2:
    Paquetes: enviados = 4, recibidos = 0, perdidos = 4
    (100% perdidos),
```

Fig. 34. ... pero en Windows el resultado es 100% de paquetes perdidos (la respuesta se pierde al volver, por la regla del punto 2).

Configuración de la regla

Se agrega la regla ACCEPT para ICMP con destino Kali, quedando como última regla del conjunto acumulado en la cadena FORWARD.

```
vboxuser@ubuntu:~$ sudo iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target    prot opt in     out     source               destination

Chain FORWARD (policy ACCEPT 111 packets, 5854 bytes)
  pkts bytes target    prot opt in     out     source               destination
 53  2826 DROP     icmp -- * *      0.0.0.0/0            192.168.70.130
  0     0 DROP     tcp  -- * *      0.0.0.0/0            192.168.70.130    tcp dpt:8080
  2    120 ACCEPT   tcp  -- * *      0.0.0.0/0            192.168.70.130    tcp dpt:22
 36  1544 DROP     tcp  -- * *      0.0.0.0/0            192.168.70.130
  5    260 DROP     tcp  -- * *      0.0.0.0/0            192.168.70.2      tcp dpt:21
 25  1475 DROP     udp  -- * *      0.0.0.0/0            192.168.70.2      udp dpt:53
  0     0 ACCEPT   icmp -- * *      0.0.0.0/0            192.168.70.2

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target    prot opt in     out     source               destination
```

Fig. 35. Estado final de la cadena FORWARD con las seis reglas acumuladas.

Después de la regla y hallazgo final

Al repetir el ping Windows→Kali, la captura en el segmento de Kali es prácticamente idéntica a la de “antes de la regla”: los cuatro pares de Echo request/reply se completan igual. Esto tiene sentido porque la política por defecto de FORWARD es ACCEPT y no existía ninguna regla previa que bloqueara ICMP con destino a Kali; el nuevo ACCEPT no cambia ningún comportamiento observable, solo deja constancia explícita de una situación que ya ocurría por defecto.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.70.130	192.168.70.2	ICMP	74	Echo (ping) request id=0x0001, seq=61/15616, ttl=127 (reply in 4)
2	0.000165174	PCSSystemtec_a9:be::	Broadcast	ARP	42	who has 192.168.70.1? Tell 192.168.70.2
3	0.001000085	PCSSystemtec_b4:60::	PCSSystemtec_a9:be::	ARP	60	192.168.70.1 is at 08:00:27:b4:60:c7
4	0.001011709	192.168.70.2	192.168.70.130	ICMP	74	Echo (ping) reply id=0x0001, seq=61/15616, ttl=64 (request in 1)
5	4.591695586	192.168.70.130	192.168.70.2	ICMP	74	Echo (ping) request id=0x0001, seq=62/15872, ttl=127 (reply in 6)
6	4.591716630	192.168.70.2	192.168.70.130	ICMP	74	Echo (ping) reply id=0x0001, seq=62/15872, ttl=64 (request in 5)
7	5.294260389	PCSSystemtec_b4:60::	PCSSystemtec_a9:be::	ARP	60	who has 192.168.70.2? Tell 192.168.70.1
8	5.294273272	PCSSystemtec_a9:be::	PCSSystemtec_b4:60::	ARP	42	192.168.70.2 is at 08:00:27:a9:be:b9
9	9.575541300	192.168.70.130	192.168.70.2	ICMP	74	Echo (ping) request id=0x0001, seq=63/16128, ttl=127 (reply in 10)
10	9.575560023	192.168.70.2	192.168.70.130	ICMP	74	Echo (ping) reply id=0x0001, seq=63/16128, ttl=64 (request in 9)
11	14.572729135	192.168.70.130	192.168.70.2	ICMP	74	Echo (ping) request id=0x0001, seq=64/16384, ttl=127 (reply in 12)
12	14.572765062	192.168.70.2	192.168.70.130	ICMP	74	Echo (ping) reply id=0x0001, seq=64/16384, ttl=64 (request in 11)

Frame 1: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface	0000	08 00 27 a9 be b9 08 00	27 b4 60 c7 08 00 45 00	E
Ethernet II, Src: PCSSystemtec_b4:60:c7 (08:00:27:b4:60:c7), Dst: PCSSystemtec_a9:be::	0010	00 3c 37 38 00 00 7f 01	f0 b3 c0 a8 46 82 c0 a8	<78.....F
Internet Protocol Version 4, Src: 192.168.70.130, Dst: 192.168.70.2	0020	46 02 08 00 4d 1e 00 01	00 3d 61 62 63 64 65 66	F..M....=abedef
Internet Control Message Protocol	0030	67 68 69 6a 6b 6c 6d 6e	6f 70 71 72 73 74 75 76	ghijklmn opqrstuv
	0040	77 61 62 63 64 65 66 67	68 69	wabdefg hi

Fig. 36. Después de la regla: el ping Windows→Kali se ve exactamente igual que antes — la regla ACCEPT no cambia nada porque ya estaba permitido por defecto.

Para confirmar que el verdadero factor limitante seguía siendo la regla del punto 2, se probó un ping directo en sentido Kali→Windows: el resultado fue 100% de paquetes perdidos (6 enviados, 0 recibidos), exactamente lo esperado, ya que cualquier ICMP con destino 192.168.70.130 sigue siendo descartado por esa regla, sin relación alguna con lo configurado en este punto.

```
(kali@kali)-[~]
$ ping 192.168.70.130
PING 192.168.70.130 (192.168.70.130) 56(84) bytes of data.
^C
— 192.168.70.130 ping statistics —
6 packets transmitted, 0 received, 100% packet loss, time 5106ms
```

Fig. 37. Ping directo Kali→Windows: 100% de pérdida, causado por la regla del punto 2 (no por nada relacionado con el punto 6).

Este resultado cierra el laboratorio con una lección clara: iptables filtra estrictamente según los criterios indicados en cada regla (aquí, la IP de destino), sin ningún conocimiento de si el tráfico pertenece a una conversación en curso ni de qué extremo la inició. Por eso una misma regla DROP puede bloquear tanto un ping iniciado por Kali como la respuesta a un ping iniciado por Windows, y por eso una regla ACCEPT dirigida a Kali no tiene ningún efecto sobre el tráfico cuyo destino es Windows.

VI. Tabla resumen de las seis reglas

#	Nodo destino	Regla (comando iptables)	Resultado esperado	Resultado observado
1	Windows (.130)	-A FORWARD -p tcp -d 192.168.70.130 --dport 8080 -j DROP	Puerto 8080 deja de responder (filtered)	Confirmado: RST -> silencio (nmap closed -> filtered)
2	Windows (.130)	-A FORWARD -p icmp -d 192.168.70.130 -j DROP	Ping hacia Windows deja de recibir respuesta	Confirmado; también bloquea el reply cuando Windows inicia el ping
3	Windows (.130)	-A FORWARD -p tcp -d 192.168.70.130 --dport 22 -j ACCEPT -A FORWARD -p tcp -d 192.168.70.130 -j DROP	Solo el 22 recibe respuesta (RST); el resto se filtra	Confirmado: 22 closed (llega), 8081 filtered
4	Kali (.2)	-A FORWARD -p tcp -d 192.168.70.2 --dport 21 -j DROP	Puerto 21 deja de responder	Confirmado: TCP falla; texto de PowerShell no distingue closed/filtered
5	Kali (.2)	-A FORWARD -p udp -d 192.168.70.2 --dport 53 -j DROP	Las consultas DNS no llegan a Kali	Confirmado: nada llega a Kali; sí llega hasta Ubuntu
6	Kali (.2)	-A FORWARD -p icmp -d 192.168.70.2 -j ACCEPT	Ping hacia Kali funciona sin cambios	Sin cambio observable (ya estaba permitido); el ping Kali→Windows sigue bloqueado por la regla 2

VII. Conclusiones generales

- La topología con Ubuntu conectando dos subredes /25 obliga a que todo el tráfico entre Kali y Windows pase por su cadena FORWARD, que es exactamente donde debe filtrarse cuando el firewall actúa como router intermedio (y no en INPUT/OUTPUT, que gobiernan el tráfico propio de Ubuntu).
- Existe una diferencia real entre un puerto “cerrado” (el host responde con RST en TCP o Port Unreachable en UDP/ICMP) y un puerto “filtrado” (DROP: silencio total y espera hasta el timeout). Herramientas como nmap (closed vs. filtered) y Wireshark (presencia o ausencia de RST/ICMP unreachable) son indispensables para diferenciarlos, porque el mensaje de error de utilidades como Test-NetConnection o nslookup puede verse igual en ambos casos.
- Una regla DROP basada únicamente en la IP de destino (-d) bloquea todo el tráfico dirigido a esa IP, sin distinguir si es una solicitud nueva o la respuesta a algo iniciado por el otro extremo. Esto se evidenció varias veces: la regla del punto 2 no solo bloquea los pings que Kali envía a Windows, sino también las respuestas que Kali genera cuando es Windows quien pregunta primero.

- Una regla ACCEPT solo cambia el comportamiento observable si existía antes alguna restricción (una política DROP o una regla DROP previa que la ACCEPT deba “adelantar”). El punto 6 mostró que agregar un ACCEPT sobre tráfico que ya pasaba por defecto no produce ningún cambio visible.
- El orden de las reglas importa: en el punto 3, el ACCEPT del puerto 22 debía colocarse antes que el DROP genérico de TCP, ya que iptables aplica la primera regla que coincide con el paquete.
- Las pruebas de “no interferencia” confirmaron en todos los casos que filtrar por -d (dirección de destino) mantiene el aislamiento esperado: las reglas dirigidas a Windows nunca afectaron el tráfico hacia Kali, y viceversa.